

Introduction to Deep Learning

Session 8: Classification & Maximum Likelihood

May 2026

Magda Gregorová - magda.gregorova@thws.de



Licensed according to CC-BY-SA 4.0

Lecture Overview

1. Classification Problem
2. Binary Classification
3. Multiclass Classification
4. Maximum Likelihood
5. Information Theory
6. Practical Implementation
7. Advanced Considerations

Classification Problem

Classification: predict discrete category $y \in \{1, \dots, K\}$

- Output: image class, spam or not, medical diagnosis
- Output space fundamentally different from regression - no notion of distance between classes

Natural objective: accuracy - fraction of correctly classified examples

$$\text{acc} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}^{(i)} = y^{(i)}]$$

From Accuracy to Loss

Problem: accuracy not differentiable - cannot use as training loss

$$\mathbf{1}[\hat{y} = y] = \begin{cases} 1 & \hat{y} = y \\ 0 & \hat{y} \neq y \end{cases} \Rightarrow \frac{\partial}{\partial \theta} \mathbf{1}[\hat{y} = y] = 0 \text{ almost everywhere}$$

Solution: differentiable proxy loss $\rightarrow$ optimise loss in training, evaluate accuracy at test

Requirements for proxy:

- Differentiable - gradient descent can optimise
- Correlates with accuracy - improving loss should improve accuracy
- Computable from NN outputs

Problem: raw network output $f_{\theta}(\mathbf{x}) \in \mathbb{R}$ cannot be interpreted as class directly

Idea: convert output to probability distribution over classes

Requirements for valid probability distribution over K classes:

$$p_k \geq 0 \quad \forall k \quad \text{and} \quad \sum_{k=1}^K p_k = 1$$

Raw network outputs (logits) $\mathbf{z} = f_{\theta}(\mathbf{x}) \in \mathbb{R}^K$

Solution: transformation that maps logits to valid probability distribution

- **Binary classification** ($K = 2$): **sigmoid** $\rightarrow$ next section
- **Multiclass classification** ($K > 2$): **softmax** $\rightarrow$ following section

Loss function: for probability outputs? not MSE, hm ...

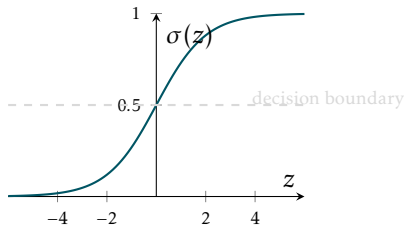
Binary Classification

Binary classification: $y \in \{0, 1\}$ - network outputs single logit $z = f_{\theta}(\mathbf{x}) \in \mathbb{R}$

Sigmoid: squashes logit to $(0, 1)$ - interpreted as probability of class 1

$$\hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- $\sigma(z) \in (0, 1)$ for all $z \in \mathbb{R}$
- $\sigma(0) = 0.5$ - symmetric around origin
- $\sigma(z) \rightarrow 1$ as $z \rightarrow \infty$
- $\sigma(z) \rightarrow 0$ as $z \rightarrow -\infty$



Binary Cross-Entropy

Loss: penalise confident wrong predictions

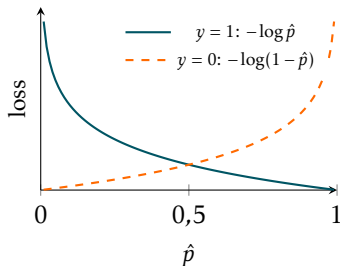
$$\ell_{BCE}(\hat{p}, y) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p})$$

$y = 1$: loss = $-\log \hat{p}$

- $\hat{p} \rightarrow 1$: loss $\rightarrow 0$ ✓
- $\hat{p} \rightarrow 0$: loss $\rightarrow \infty$ ✗

$y = 0$: loss = $-\log(1 - \hat{p})$

- $\hat{p} \rightarrow 0$: loss $\rightarrow 0$ ✓
- $\hat{p} \rightarrow 1$: loss $\rightarrow \infty$ ✗



Training loss:

$$\mathcal{L}_{BCE}(\theta) = -\frac{1}{N} \sum_{i=1}^N \left[y^{(i)} \log \hat{p}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]$$

Multiclass Classification

Multiclass classification: $y \in \{1, \dots, K\}$ - network outputs K logits $\mathbf{z} \in \mathbb{R}^K$

Softmax: maps logits to valid probability distribution

$$\hat{p}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad k = 1, \dots, K$$

- $\hat{p}_k > 0$ for all k - all probabilities positive
- $\sum_{k=1}^K \hat{p}_k = 1$ - valid distribution
- Larger logit $\Rightarrow$ larger probability

Connection to sigmoid: softmax with $K = 2$ reduces to sigmoid

$$\hat{p}_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}} = \frac{1}{1 + e^{-(z_1 - z_2)}} = \sigma(z_1 - z_2)$$

Single logit $z = z_1 - z_2$ - one degree of freedom is redundant

Categorical Cross-Entropy

Labels: one-hot vector $\mathbf{y} \in \{0, 1\}^K$ - $y_k = 1$ for correct class, 0 otherwise

Loss: picks out log probability of correct class

$$\ell_{CE}(\hat{\mathbf{p}}, \mathbf{y}) = - \sum_{k=1}^K y_k \log \hat{p}_k = -\log \hat{p}_y$$

- Correct class predicted confidently $\hat{p}_y \rightarrow 1$: loss $\rightarrow 0$ ✓
- Correct class predicted with low confidence $\hat{p}_y \rightarrow 0$: loss $\rightarrow \infty$ ✗

Training loss:

$$\mathcal{L}_{CE}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log \hat{p}_{y^{(i)}}^{(i)}$$

Maximum Likelihood

Maximum Likelihood Principle

Setup: model defines conditional distribution $p(y | \mathbf{x}; \theta)$ over outputs

Likelihood: probability of observed data as a **function of parameters** - data $\mathcal{D}$ is fixed

Assumption: examples i.i.d. - joint likelihood factorises as product over examples

$$L(\theta) = p(\mathcal{D}; \theta) = \prod_{i=1}^N p(y^{(i)} | \mathbf{x}^{(i)}; \theta)$$

MLE: find parameters that make observed data most probable

$$\theta^* = \arg \max_{\theta} L(\theta)$$

Log-likelihood: sum instead of product - same maximiser, more convenient

$$\log L(\theta) = \sum_{i=1}^N \log p(y^{(i)} | \mathbf{x}^{(i)}; \theta)$$

Maximise log-likelihood $\equiv$ **minimise** negative log-likelihood (NLL)

$$\theta^* = \arg \max_{\theta} \log L(\theta) = \arg \max_{\theta} \sum_{i=1}^N \log p(y^{(i)} | \mathbf{x}^{(i)}; \theta) = \arg \min_{\theta} \underbrace{-\frac{1}{N} \sum_{i=1}^N \log p(y^{(i)} | \mathbf{x}^{(i)}; \theta)}_{\mathcal{L}(\theta)}$$

Key insight: choice of $p(y | \mathbf{x}; \theta)$ determines the loss function

Distributional assumption	Loss function
Gaussian	MSE
Bernoulli	Binary cross-entropy
Categorical	Categorical cross-entropy

Gaussian Assumption $\rightarrow$ MSE

Assume: $p(y | \mathbf{x}; \boldsymbol{\theta}) = \mathcal{N}(y; f_{\boldsymbol{\theta}}(\mathbf{x}), \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - f_{\boldsymbol{\theta}}(\mathbf{x}))^2}{2\sigma^2}\right)$

Log-likelihood: $\log p(y | \mathbf{x}; \boldsymbol{\theta}) = -\frac{(y - f_{\boldsymbol{\theta}}(\mathbf{x}))^2}{2\sigma^2} - \frac{1}{2} \log(2\pi\sigma^2)$

NLL:
$$-\frac{1}{N} \sum_{i=1}^N \log p(y^{(i)} | \mathbf{x}^{(i)}; \boldsymbol{\theta}) = \frac{1}{2\sigma^2} \underbrace{\frac{1}{N} \sum_{i=1}^N (y^{(i)} - f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}))^2}_{\text{MSE}} + \text{const}$$

Minimising NLL under Gaussian assumption $\equiv$ minimising **MSE** - **MSE not arbitrary!**

$$\arg \min_{\boldsymbol{\theta}} NLL_{\mathcal{N}}(\boldsymbol{\theta}) = \arg \min_{\boldsymbol{\theta}} MSE(\boldsymbol{\theta})$$

Bernoulli Assumption $\rightarrow$ Binary Cross-Entropy

Assume: $p(y | \mathbf{x}; \theta) = \text{Bernoulli}(y; \hat{p}) = \hat{p}^y (1 - \hat{p})^{1-y}, \quad \hat{p} = \sigma(f_{\theta}(\mathbf{x}))$

Log-likelihood: $\log p(y | \mathbf{x}; \theta) = y \log \hat{p} + (1 - y) \log(1 - \hat{p})$

$$\text{NLL:} \quad -\frac{1}{N} \sum_{i=1}^N \log p(y^{(i)} | \mathbf{x}^{(i)}; \theta) = -\frac{1}{N} \sum_{i=1}^N \underbrace{\left[y^{(i)} \log \hat{p}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]}_{\text{binary cross-entropy } \mathcal{L}_{BCE}(\theta)}$$

Minimising NLL under Bernoulli assumption $\equiv$ minimising **binary cross-entropy**

$$\arg \min_{\theta} \text{NLL}_{Be}(\theta) = \arg \min_{\theta} \mathcal{L}_{BCE}(\theta)$$

Categorical Assumption $\rightarrow$ Categorical Cross-Entropy

Assume: $p(y | \mathbf{x}; \theta) = \text{Categorical}(y; \hat{\mathbf{p}}) = \prod_{k=1}^K \hat{p}_k^{y_k} = \hat{p}_y, \quad \hat{\mathbf{p}} = \text{softmax}(f_{\theta}(\mathbf{x}))$

Log-likelihood: $\log p(y | \mathbf{x}; \theta) = \sum_{k=1}^K y_k \log \hat{p}_k = \log \hat{p}_y$

NLL:
$$-\frac{1}{N} \sum_{i=1}^N \log p(y^{(i)} | \mathbf{x}^{(i)}; \theta) = \underbrace{-\frac{1}{N} \sum_{i=1}^N \log \hat{p}_{y^{(i)}}^{(i)}}_{\mathcal{L}_{CE}(\theta)}$$

Minimising NLL under categorical assumption $\equiv$ minimising **categorical cross-entropy**

$$\arg \min_{\theta} NLL_{Cat}(\theta) = \arg \min_{\theta} \mathcal{L}_{CE}(\theta)$$

Information Theory

Why Information Theory?

So far: cross-entropy - mathematical result of MLE derivation

Open question: what does cross-entropy *mean*?

Answer - information theory:

- Cross-entropy: fundamental measure of how well distribution q approximates distribution p
- Training minimises gap between model distribution $q = \hat{p}$ and true distribution p

We will build up:

- **Information content** - how surprising is an event?
- **Entropy** - how uncertain is a distribution?
- **Cross-entropy and KL divergence** - how far apart are two distributions?

Payoff: MLE, cross-entropy loss, and KL divergence - three views of same objective

Intuition: how surprised are we when event k occurs?

- Rare event - very surprising - highly informative
- Common event - expected - low information

Information content: $I(k) = -\log_2 p_k$ - optimal number of bits to encode event k

p_k	$I(k)$	Interpretation
1	0	certain event - no bits needed
0.5	1 bit	coin flip
0.01	≈ 6.6 bits	rare event - very surprising - long encoding
$\rightarrow 0$	$\rightarrow \infty$	impossible event becoming true - impossible to encode

Why $-\log$? More probable events - shorter codes; info of independent events - sum of info; monotonic and smooth (Shannon, 1948).

Entropy: expected information content = minimum average code length for samples from distribution p

$$H(p) = \mathbb{E}_{k \sim p}[I(k)] = - \sum_k p_k \log p_k$$

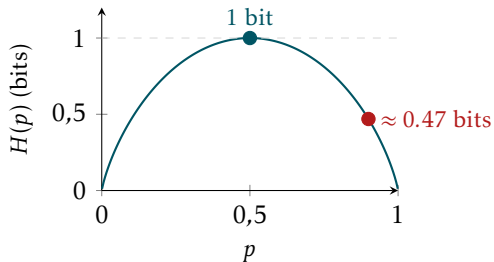
Measures uncertainty:

- High entropy - high uncertainty, long average code - distribution spread out
- Low entropy - low uncertainty, short average code - distribution concentrated
- $H(p) = 0$ - deterministic distribution, no bits needed
- $H(p)$ maximised by uniform distribution - maximum uncertainty

Binary distribution: $p = (p, 1 - p)$

$$H(p) = -p \log_2 p - (1 - p) \log_2 (1 - p)$$

- **Fair coin** $p = 0.5$: $H = 1$ bit - optimal code 1 bit per outcome; matches practice
- **Biased coin** $p = 0.9$: $H \approx 0.47$ bits - in theory we can do better than 1 bit per symbol



Practical limit: single outcomes cannot be encoded in less than 1 bit. Bound $H(p) < 1$ only achievable asymptotically by encoding longer sequences together

Cross-Entropy

Setup: true distribution p , model distribution $\hat{\mathbf{p}}$

Cross-entropy: average code length when using $\hat{\mathbf{p}}$ to encode events from p

$$H(p, \hat{\mathbf{p}}) = - \sum_k p_k \log \hat{p}_k$$

- $H(p, \hat{\mathbf{p}}) \geq H(p)$ - using wrong distribution always costs extra bits
- $H(p, \hat{\mathbf{p}}) = H(p)$ iff $p = \hat{\mathbf{p}}$ - no extra cost when model is perfect

Connection to classification loss:

- True labels $\Rightarrow$ true distribution $p \sim$ one-hot $\mathbf{y} \in \{0, 1\}^K$
- Model predictions $\Rightarrow$ estimated distribution $\hat{\mathbf{p}}_{\theta} = \text{softmax}(f_{\theta}(\mathbf{x}))$

$$\mathcal{L}_{CE}(\theta) = NLL_{CE}(\theta) = H(p, \hat{\mathbf{p}}_{\theta})$$

KL divergence: extra cost of using $\hat{p}$ instead of p

$$D_{KL}(p||\hat{p}) = H(p, \hat{p}) - H(p) = \sum_k p_k \log \frac{p_k}{\hat{p}_k} = \underbrace{\sum_k p_k \log p_k}_{-H(p)} - \underbrace{\sum_k p_k \log \hat{p}_k}_{H(p, \hat{p})}$$

- $D_{KL}(p||\hat{p}) \geq 0$ - always non-negative
- $D_{KL}(p||\hat{p}) = 0$ iff $p = \hat{p}$
- Not symmetric: $D_{KL}(p||\hat{p}) \neq D_{KL}(\hat{p}||p)$ in general

Connection to cross-entropy: $H(p)$ is fixed - labels do not depend on θ

$$\arg \min_{\theta} D_{KL}(p||\hat{p}) \equiv \arg \min_{\theta} H(p, \hat{p})$$

Min KL divergence btw true and predicted distribution $\equiv$ minimising cross-entropy
(Holds equivalently for any other distribution family.)

Three Views, One Objective

Three perspectives - same optimisation problem:

Perspective	Gaussian	Bernoulli	Categorical
Loss	MSE	binary CE	categorical CE
MLE	maximise $\log \mathcal{N}$	maximise $\log \text{Ber}$	maximise $\log \text{Cat}$
Info theory	$\min D_{KL}$	$\min D_{KL}$	$\min D_{KL}$

Choice of loss not arbitrary ~ assumption about data-generating process!

Practical Implementation

Binary classification:

- `nn.BCEWithLogitsLoss` - takes logits
- `nn.BCELoss` - takes probabilities

Multiclass classification:

- `nn.CrossEntropyLoss` - takes logits
- `nn.NLLLoss` - takes log-probabilities

Why logits?

- Numerically stable - PyTorch applies log-sum-exp trick internally
- Fewer operations - no need to apply sigmoid/softmax before loss
- Less risk of errors - sigmoid then BCELoss is a common bug source

Note: `nn.CrossEntropyLoss` expects:

- Integer class indices of shape $(N,)$ - standard usage
- Float probability distributions of shape (N, K) - for soft targets (see later)

One-hot integer vectors are **not supported**

Cross-entropy loss requires $\log \hat{p}_k$:

$$\mathcal{L} = -\frac{1}{N} \sum_i \log \hat{p}_{y^{(i)}}^{(i)} \quad \Rightarrow \quad \log \hat{p}_k = \log \frac{e^{z_k}}{\sum_j e^{z_j}} = z_k - \log \sum_j e^{z_j}$$

Problem: log-softmax numerically unstable

Large logits $\Rightarrow e^{z_k}$ overflows to inf Small logits $\Rightarrow \log \sum_j e^{z_j}$ underflows to $-\text{inf}$

Log-sum-exp trick: subtract maximum logit before exponentiating

$$\log \sum_j e^{z_j} = c + \log \sum_j e^{z_j - c}, \quad c = \max_j z_j$$

- Largest term is always $e^0 = 1$ - no overflow
- Other terms ≤ 1 - no underflow

`nn.CrossEntropyLoss` and `nn.BCEWithLogitsLoss` apply this internally!

From logits to predicted class:

- Softmax is monotone - highest logit = highest probability class
- No need to apply softmax / sigmoid
- **Multiclass:** $\hat{y} = \arg \max_k z_k$
- **Binary:** decision boundary at logit = 0 - predict class 1 if $z > 0$

Accuracy: fraction of correctly classified examples

$$\text{acc} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}^{(i)} = y^{(i)}]$$

Advanced Considerations

Problem: cross-entropy with hard labels always rewards higher confidence

$$\ell_{CE}(\hat{\mathbf{p}}, \mathbf{y}) = - \sum_{k=1}^K y_k \log \hat{p}_k = -\log \hat{p}_y \quad \text{minimised as } \hat{p}_y \rightarrow 1$$

- Rewarded for confidence - never penalised for overconfidence
- Learns to saturate softmax - assigns high probability even when uncertain
- Out-of-distribution input: may assign 99% confidence to wrong class

Information theory view: entropy $H(\hat{\mathbf{p}})$ too low - model is too certain

Predicted probabilities not indicative for true prediction (un)certainty!

Fix: replace hard one-hot targets $\mathbf{y}$ with soft targets $\mathbf{y}^{\text{smooth}}$

$$\ell_{CE}(\hat{\mathbf{p}}, \mathbf{y}^{\text{smooth}}) = - \sum_{k=1}^K y_k^{\text{smooth}} \log \hat{p}_k, \quad \text{where } y_k^{\text{smooth}} = \begin{cases} 1 - \varepsilon & k = y \\ \varepsilon / (K - 1) & k \neq y \end{cases}$$

- Model can never be 100% confident - softmax cannot saturate
- Increases entropy of predictions - more calibrated uncertainty
- Typical value: $\varepsilon = 0.1$

Standard in modern architectures: Inception, Vision Transformers, modern ResNets - consistently effective with small ε

Well-calibrated model: predicted probability = empirical accuracy

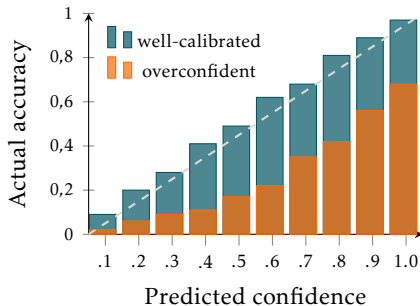
predicted confidence 80% $\Rightarrow$ correct 80% of the time

Reliability diagram:

- group data to bins by predicted confidence
- get actual accuracy of samples in bin
- well-calibrated model on diagonal

Temperature scaling: divide logits $\mathbf{z}$ by $\tau > 1$

$$\hat{\mathbf{p}} = \text{softmax}\left(\frac{\mathbf{z}}{\tau}\right); \quad \hat{p}_k = \frac{e^{z_k/\tau}}{\sum_j e^{z_j/\tau}}$$



Problem: training set has far more examples of some classes than others

- Cross-entropy dominated by frequent classes
- Model learns to ignore rare classes - predicting majority class is cheap
- Common in practice: medical diagnosis, fraud detection, rare events

Fixes:

- **Weighted loss** - upweight rare classes: $\mathcal{L} = -\frac{1}{N} \sum_i w_{y^{(i)}} \log \hat{p}_{y^{(i)}}^{(i)}$
- **Oversampling** - duplicate rare class examples
- **Undersampling** - remove frequent class examples

Always check class distribution before training

Labels are often imperfect in practice:

- **Noisy labels** - annotators make mistakes; loss computed against wrong targets
- **Partial labels** - only some examples labelled; missing labels treated as negatives
- **Ambiguous labels** - multiple valid answers; hard labels force arbitrary choice

Consequence: model optimises toward label, not the truth!

Mitigations:

- Label cleaning - detect and correct noisy labels
- Label smoothing - reduces impact of individual wrong labels
- Multiple annotators - aggregate labels to reduce noise

Label quality matters as much as model quality